OCOM5300M Artificial Intelligence Project Calibration Set

In [1]:
```python
# Deep learning Functionality
import torch  # PyTorch library

# Data Handling And Analysis
import pandas as pd  # data manipulation and analysis using DataFrame structures
from sklearn.model_selection import train_test_split  # splitting data into training and test set

# Utilities
import os  # file and directory operations
from pathlib import Path # file and directory paths
import shutil # high-level file operations such as copying, moving, renaming, and deleting files or folders
```

In [2]:
```python
# suppress Intel specific Math Kernel Library (MKL) warnings
os.environ['KMP_DUPLICATE_LIB_OK'] = 'True'
```

In [3]:
```python
# detect device (CPU/GPU)
device = torch.device('cpu')
```

In [4]:
```python
# display the complete content of DataFrame columns without truncation
pd.set_option('display.max_colwidth', None);
```

Split Original ROCOv2 Captions Into Validation And Calibration Sets

In [5]:
```python
# valid_concepts_manual.csv contains the manually curated UMLS CUIs (Unified Medical Language System Concept Unique Identifier)
# associated with each ROCOv2 validation image, identifying imaging modality and additional anatomical or directional information for X-Rays
valid_concepts_manual = pd.read_csv("ROCOV2/valid_concepts_manual.csv")
```

In [6]:
```python
# maps each UMLS CUI to the name of its medical concept (canonical name)
cui_mapping = pd.read_csv("ROCOV2/cui_mapping.csv")
```

In [7]:
```python
# extract modalities, i.e. medical imaging technique used to create the image
cui_modalities = (
    valid_concepts_manual["CUIs"]
```

```
    .dropna()
    .str.split(";")
    .str[0]
    .unique()
)
```

In [8]:
```
# map each modality to its medical concept (canonical name)
modality_mapping = (
    cui_mapping[
        cui_mapping["CUI"].isin(cui_modalities)
    ]
    .set_index("CUI")["Canonical name"]
    .to_dict()
)
```

In [9]:
```
# display mappings
modality_mapping
```

Out[9]:
```
{'C0032743': 'Positron-Emission Tomography',
 'C0024485': 'Magnetic Resonance Imaging',
 'C0002978': 'angiogram',
 'C0040405': 'X-Ray Computed Tomography',
 'C0041618': 'Ultrasonography',
 'C1306645': 'Plain x-ray'}
```

In [10]:
```
# convert semicolon-separated list of CUIs into a single imaging modality label
def extract_modality(cuis):

    # assign category combined modality (DRCO), i.e. image supports multiple imaging techniques
    # ROCOv2 defines DRCO (combined modality) as images combining mulitple supported imaging techniques, e.g., PET-CT, PET-MRI, SPECT-CT, or SPECT-MRI
    if pd.isna(cuis) or str(cuis).strip() == "":
        return "Combined modalities"

    # extract CUI codes from semicolon-separated CUI string
    cui_set = set(str(cuis).split(";"))

    # comprehension list indicating which known modalities are in the set of cuis
    matches = [
        modality
        for cui, modality in modality_mapping.items()
```

```
        if cui in cui_set
    ]

    # return unknown modality CUI
    if len(matches) == 0:
        return "Unknown"

    # return modality
    if len(matches) == 1:
        return matches[0]

    # return multiple modalities
    return "Multiple: " + ", ".join(matches)
```

In [11]:
```
# run function on each value in the pandas series
valid_concepts_manual["modality"] = valid_concepts_manual["CUIs"].apply(extract_modality)
```

In [12]:
```
# display count and percentage of each modality in the original validation set
modality_table = pd.DataFrame({
    "Count": valid_concepts_manual["modality"].value_counts(),
    "Percentage": (
        valid_concepts_manual["modality"]
        .value_counts(normalize=True)
        .mul(100)
        .round(2)
    )
})

modality_table
```

Out[12]:

modality	Count	Percentage
X-Ray Computed Tomography	3673	37.09
Plain x-ray	2670	26.96
Ultrasonography	1528	15.43
Magnetic Resonance Imaging	1444	14.58
angiogram	421	4.25
Combined modalities	130	1.31
Positron-Emission Tomography	38	0.38

In [13]:
```
# 50:50 stratified split for valid_concepts_manual
dfValidation, dfCalibration = train_test_split(
    valid_concepts_manual,
    test_size=0.50,
    random_state=1,
    shuffle=True,
    stratify=valid_concepts_manual["modality"]
)
```

In [14]:
```
# reset indexes
dfValidation = dfValidation.reset_index(drop=True)
```

In [15]:
```
# display validation split modality counts
validation_modality_counts = (
    dfValidation["modality"]
    .value_counts()
    .sort_index()
    .rename_axis("Modality")
    .reset_index(name="Count")
)

validation_modality_counts.loc[len(validation_modality_counts)] = [
    "Total",
```

```
    validation_modality_counts["Count"].sum()
]

display(validation_modality_counts)
```

	Modality	Count
0	Combined modalities	65
1	Magnetic Resonance Imaging	722
2	Plain x-ray	1335
3	Positron-Emission Tomography	19
4	Ultrasonography	764
5	X-Ray Computed Tomography	1836
6	angiogram	211
7	Total	4952

In [16]:
```
# reset indexes
dfCalibration = dfCalibration.reset_index(drop=True)
```

In [17]:
```
# display calibration split modality counts
calibration_modality_counts = (
    dfCalibration["modality"]
    .value_counts()
    .sort_index()
    .rename_axis("Modality")
    .reset_index(name="Count")
)

calibration_modality_counts.loc[len(calibration_modality_counts)] = [
    "Total",
    calibration_modality_counts["Count"].sum()
]

display(calibration_modality_counts)
```

	Modality	Count
0	Combined modalities	65
1	Magnetic Resonance Imaging	722
2	Plain x-ray	1335
3	Positron-Emission Tomography	19
4	Ultrasonography	764
5	X-Ray Computed Tomography	1837
6	angiogram	210
7	Total	4952

In [18]:
```
# load valid captions
valid_captions = pd.read_csv("ROCOv2/valid_captions.csv")
```

In [19]:
```
# extract Ids and captions for new validation split
dfValidationCaptions = valid_captions[
    valid_captions["ID"].isin(dfValidation["ID"])
].copy()
```

In [20]:
```
# reset index
dfValidationCaptions = dfValidationCaptions.reset_index(drop=True)
```

In [21]:
```
# create new valid_split csv file
dfValidationCaptions.to_csv("ROCOv2/valid_captions_split.csv", index=False)
```

In [22]:
```
# extract Ids and captions for new calibration split
dfCalibrationCaptions = valid_captions[
    valid_captions["ID"].isin(dfCalibration["ID"])
].copy()
```

In [23]:
```
# reset index
dfCalibrationCaptions = dfCalibrationCaptions.reset_index(drop=True)
```

In [24]:
```
# create new calibration_split csv file
dfCalibrationCaptions.to_csv("ROCOv2/calibration_captions_split.csv", index=False)
```

Split Original ROCOv2 Images Into Validation And Calibration Sets

In [25]:
```
# original ROCOv2 validation images
image_folder = Path("ROCOv2/valid_images/valid")
```

In [26]:
```
# original ROCOv2 validation images
validation_folder = Path("ROCOv2/valid_images/validation")
```

In [27]:
```
# validation image ids
validation_ids = set(dfValidation["ID"].astype(str))
```

In [29]:
```
# copy valid images from validation folder to revised validation folder
for image_path in image_folder.iterdir():

    if (
        image_path.is_file()
        and image_path.suffix.lower() == ".jpg"
        and image_path.stem in validation_ids
    ):
        shutil.copy2(
            str(image_path),
            str(validation_folder / image_path.name)
        )
```

In [30]:
```
# new folder for conformal calibration images
calibration_folder = Path("ROCOv2/calibration_images/calibration")
```

In [31]:
```
# calibration image ids
calibration_ids = set(dfCalibration["ID"].astype(str))
```

In [32]:
```
# copy valid images from validation folder to calibration folder
for image_path in image_folder.iterdir():

    if (
        image_path.is_file()
```

```
        and image_path.suffix.lower() == ".jpg"
        and image_path.stem in calibration_ids
    ):
        shutil.copy2(
            str(image_path),
            str(calibration_folder / image_path.name)
        )
```

In [33]:
```
num_files = sum(1 for f in validation_folder.iterdir() if f.is_file())
print(f"Number of files: {num_files}")
```

```
Number of files: 4952
```

In [34]:
```
num_files = sum(1 for f in calibration_folder.iterdir() if f.is_file())
print(f"Number of files: {num_files}")
```

```
Number of files: 4952
```

References

ROCOv2: Radiology Objects in Context Version 2, An Updated Multimodal Image Dataset https://zenodo.org/records/10821435

RadImageNet_pytorch.zip for ResNet50.pt learned parameter tensors
https://drive.google.com/file/d/1RHt2GnuOYIc_gcoTETtBDSW73mFyRAtR/view
RadImageNet Official Code And Document Repository https://github.com/BMEII-AI/RadImageNet

OCOM5100M Programming For Data Science

OCOM5101M Data Science

OCOM5200M Machine Learning

OCOM5203M Deep Learning

OCOM5203M Deep Learning Module, Summative Assessment 2

OCOM5204M Data Mining And Analytics

Project Jupyter, Jupyter Notebook documentation. Available At: https://jupyter-notebook.readthedocs.io/en/stable/

Python Software Foundation, Python 3.13 documentation (2026). Available At: https://docs.python.org/3.13/